

Bölüm 3 — Sözdizimi ve Anlambilim (Chapter 3)

Programlama Dilleri

Sözdizimi ve Anlambilim Tanımlaması

Tüm Sorular — İngilizce & Türkçe + Parse Tree Diyagramları

Kaynak Kitap:

Concepts of Programming Languages

Robert W. Sebesta — 12. Baskı

29

Gözden Geçirme
Sorusu

28

Problem
Seti Sorusu

57

Toplam
Soru

**Parse
Tree**
Ağaç
Diyagramları

- Gözden Geçirme Soruları (Review Questions)
- Problem Seti (Problem Set)
- Parse Tree / Türetme Diyagramı içeren sorular

BÖLÜM 3 — GÖZDEN GEÇİRME SORULARI (Review Questions) • 29 Soru**Soru 1**

EN: Define syntax and semantics.

TR: Sözdizimi (syntax) ve anlambilimi (semantics) tanımlayın.

● Cevap / Yönlendirme

Sözdizimi (syntax), bir programlama dilindeki ifadelerin, deyimlerin ve program birimlerinin biçimini — nasıl yazılmaları gerektiğini — tanımlar. Anlambilim (semantics) ise bu biçimsel yapıların anlamını, yani programın çalıştığında ne yapacağını tanımlar. Sözdizimi kurallara uygunlukla, anlambilim ise doğrulukla ilgilidir.

Soru 2

EN: Who are language descriptions for?

TR: Dil tanımlamaları kimin içindir?

● Cevap / Yönlendirme

Dil tanımlamaları iki ana kitle için hazırlanır: (1) Programcılar — dili kullanarak program yazan kişiler; dili doğru kullanabilmek için sözdizimi ve anlambilim kurallarına ihtiyaç duyarlar. (2) Derleyici/yorumlayıcı yazarları (implementors) — dil işlemcilerini geliştiren kişiler; kesin ve tutarsız olmayan bir tanıma ihtiyaç duyarlar.

Soru 3

EN: Describe the operation of a general language generator.

TR: Genel bir dil üreticinin (language generator) işleyişini açıklayın.

● Cevap / Yönlendirme

Dil üreteç (language generator), bir dilin tüm olası geçerli cümlelerini üretebilen bir sistemdir. Üretimler (productions) adı verilen kurallar kümesinden başlayarak, başlangıç sembolünden (start symbol) yola çıkıp kural uygulamalarıyla terminal semboller içeren geçerli dizeler oluşturur. BNF ve bağlamdan bağımsız gramerler (context-free grammars) bu türün en yaygın örnekleridir.

Soru 4

EN: Describe the operation of a general language recognizer.

TR: Genel bir dil tanıyıcının (language recognizer) işleyişini açıklayın.

● Cevap / Yönlendirme

Dil tanıyıcı (language recognizer), verilen bir girdi dizisinin belirli bir dilin parçası olup olmadığını belirleyen bir sistemdir. Gelen deyim ya da cümleyi dil gramerine göre ayrıştırır (parse eder); geçerliyse kabul eder, değilse hata bildirir. Derleyicilerin sözdizimi analiz (syntax analysis / parsing) aşaması bu türün pratik uygulamasıdır.

Soru 5

EN: What is the difference between a sentence and a sentential form?

TR: Cümle (sentence) ile cümlesel form (sentential form) arasındaki fark nedir?

● Cevap / Yönlendirme

Cümlesel form (sentential form), bir türetme sürecinin herhangi bir adımında elde edilen — hem terminal hem de non-terminal semboller içerebilen — dizedir. Cümle (sentence) ise yalnızca terminal sembollerden oluşan, türetme sürecinin sonunda ulaşılan özel bir cümlesel formdur. Kısacası her cümle bir cümlesel formdur, ancak her cümlesel form cümle değildir.

Soru 6

EN: Define a left-recursive grammar rule.

TR: Sol özyinelemeli (left-recursive) bir gramer kuralını tanımlayın.

● Cevap / Yönlendirme

Sol özyinelemeli (left-recursive) bir gramer kuralı, bir non-terminal sembolün kendi kendine doğrudan veya dolaylı olarak türetmenin en solunda yer aldığı kuraldır. Örnek: $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$. Bu kurala göre $\langle \text{expr} \rangle$ 'nin en soldaki sembol olarak yeniden $\langle \text{expr} \rangle$ üretmesi, bazı ayrıştırma algoritmalarının (özellikle top-down parsers) sonsuz döngüye girmesine yol açabilir.

Soru 7

EN: What three extensions are common to most EBNFs?

TR: Çoğu EBNF'de yaygın olan üç eklenti nedir?

● Cevap / Yönlendirme

Çoğu EBNF'de (Extended Backus-Naur Form) yer alan üç temel eklenti: (1) Köşeli parantez $[]$ — isteğe bağlı (optional) öğeleri gösterir; (2) Küme parantezi $\{ \}$ — sıfır veya daha fazla tekrarı (zero or more repetitions) gösterir; (3) Yuvarlak parantez içinde dikey çizgi $(|)$ — seçenekler arasındaki alternatif gösterir. Bu eklentiler BNF tanımlamalarını çok daha kısa ve okunabilir kılar.

Soru 8

EN: Distinguish between static and dynamic semantics.

TR: Statik (static) ve dinamik (dynamic) anlambilim arasındaki farkı açıklayın.

● Cevap / Yönlendirme

Statik anlambilim (static semantics), çalışma zamanından önce — genellikle derleme aşamasında — kontrol edilebilen kurallardır. Tür uyumluluğu (type compatibility), değişkenlerin bildirilmeden kullanılmaması ve parametre sayısı eşleşmeleri bu kategoridedir. Dinamik anlambilim (dynamic semantics) ise programın çalışma zamanındaki (runtime) davranışını tanımlar; deyimlerin yürütüldüğünde ne anlama geldiğini belirler.

Soru 9

EN: What purpose do predicates serve in an attribute grammar?

TR: Öznitelik gramerinde (attribute grammar) yüklemeler (predicates) ne işe yarar?

● Cevap / Yönlendirme

Öznitelik gramerinde yüklemeler (predicates), statik anlambilim kurallarını denetlemek için kullanılır. Bir yükleme, bir ayrıştırma ağacı düğümünün öznitelik değerlerinin sağlaması gereken koşulları ifade eden boolean ifadelerdir. Örneğin tür uyumluluğunu kontrol eden bir yüklem, atamanın sol tarafı ile sağ tarafının aynı türde olmasını zorunlu kılabilir.

Soru 10

EN: What is the difference between a synthesized and an inherited attribute?

TR: Sentezlenmiş (synthesized) ve miras alınan (inherited) öznitelik arasındaki fark nedir?

● Cevap / Yönlendirme

Sentezlenmiş öznitelik (synthesized attribute), bir ayrıştırma ağacı düğümünün öznitelik değerinin yalnızca o düğümün alt çocuklarından (children) hesaplanmasıdır — bilgi aşağıdan yukarıya akar. Miras alınan öznitelik (inherited attribute) ise bir düğümün değerinin üst düğümünden (parent) ya da kardeş düğümlerinden (siblings) türetilmesidir — bilgi yukarıdan aşağıya akar.

Soru 11

EN: How is the order of evaluation of attributes determined for the trees of a given attribute grammar?

TR: Belirli bir öznitelik gramerinin ağaçları için özniteliklerin değerlendirme sırası nasıl belirlenir?

● Cevap / Yönlendirme

Özniteliklerin değerlendirme sırası, öznitelik bağımlılık grafi (attribute dependency graph) kullanılarak belirlenir. Bu grafta düğümler öznitelikleri, yönlü kenarlar ise bağımlılıkları temsil eder. Döngüsüz (acyclic) bu grafin topolojik sıralaması (topological sort) yapılarak özniteliklerin hangi sırada hesaplanması gerektiği bulunur. Döngü varsa öznitelik grameri hatalıdır.

Soru 12

EN: What is the primary use of attribute grammars?

TR: Öznitelik gramerlerinin (attribute grammars) birincil kullanım amacı nedir?

● Cevap / Yönlendirme

Öznitelik gramerleri öncelikle programlama dillerinin statik anlambilimini (static semantics) tanımlamak ve kontrol etmek için kullanılır. Derleyicilerin anlambilim analizi (semantic analysis) aşamasında tür kontrolü, kapsam kuralları ve bağlanma (binding) analizi gibi işlemleri gerçekleştirmek için teorik temel sağlarlar. Ayrıca derleyici üretici araçlarda (parser generators) pratik uygulama alanı bulurlar.

Soru 13

EN: Explain the primary uses of a methodology and notation for describing the semantics of programming languages.

TR: Programlama dillerinin anlambilimini tanımlamaya yönelik metodoloji ve gösterimin birincil kullanımlarını açıklayın.

● Cevap / Yönlendirme

Anlambilim tanımlama metodolojilerinin başlıca kullanımları: (1) Dil tasarımcıları için — tutarsızlıkları erken tespit etmek ve dilin davranışını kesin biçimde tanımlamak; (2) Derleyici yazarları için — doğru implementasyon için kesin referans; (3) Program doğrulama (program verification) için — programların doğruluğunu matematiksel olarak kanıtlamak; (4) Dil öğrenenleri için — dilin davranışını kesin biçimde anlamak.

Soru 14

EN: Why can machine languages not be used to define statements in operational semantics?

TR: İşlemsel anlambilimde (operational semantics) deyimleri tanımlamak için neden makine dilleri kullanılamaz?

● Cevap / Yönlendirme

Makine dilleri platforma özel olduğundan dil tanımını belirli bir donanıma bağlar ve taşınabilirliği ortadan kaldırır. Aynı zamanda makine dili deyimleri son derece düşük seviyeli olduğundan, üst düzey dil yapılarının anlamını açıklamak yerine ek karmaşıklık getirir. Bu nedenle işlemsel anlambilimde sanal makineler (virtual machines) veya idealize edilmiş hesaplama modelleri kullanılır.

Soru 15

EN: Describe the two levels of uses of operational semantics.

TR: İşlemsel anlambilimin (operational semantics) iki kullanım düzeyini açıklayın.

● Cevap / Yönlendirme

İşlemsel anlambilim iki farklı düzeyde kullanılır: (1) Doğal anlambilim (natural / big-step semantics) — bir ifade ya da programın son durumunu (final state) doğrudan tanımlar; programın bütünsel davranışına odaklanır. (2) Yapısal işlemsel anlambilim (structural / small-step semantics) — her hesaplama adımını tek tek tanımlar; programın nasıl çalıştığının adım adım izlenmesine olanak tanır.

Soru 16

EN: In denotational semantics, what are the syntactic and semantic domains?

TR: Gösterimsel anlambilimde (denotational semantics) sözdizimsel ve anlabilimsel alanlar nelerdir?

● Cevap / Yönlendirme

Gösterimsel anlambilimde sözdizimsel alan (syntactic domain), dilin sözdizimi tarafından tanımlanan yapıların kümesidir — program metni, ifadeler, deyimler vb. Anlabilimsel alan (semantic domain) ise bu sözdizimsel yapıların eşlendiği matematiksel nesnelerin kümesidir; genellikle tamsayılar, doğruluk değerleri, fonksiyonlar veya durum uzayları (state spaces) bu alanı oluşturur.

Soru 17

EN: What is stored in the state of a program for denotational semantics?

TR: Gösterimsel anlambilim için bir programın durumunda (state) neler saklanır?

● Cevap / Yönlendirme

Gösterimsel anlambilimde program durumu (program state), değişkenleri değerlere eşleyen bir fonksiyon olarak modellenir. Bu fonksiyon, her değişken adını (veya bellek konumunu) o andaki değerine bağlar. Durum aynı zamanda tanımsız (undefined) değerleri de içerebilir; bu, henüz atanmamış değişkenleri temsil eder.

Soru 18

EN: Which semantics approach is most widely known?

TR: En yaygın bilinen anlambilim yaklaşımı hangisidir?

● Cevap / Yönlendirme

Aksiomatik anlambilim (axiomatic semantics), özellikle program doğrulama (program verification) alanındaki teorik önemi nedeniyle en yaygın bilinen yaklaşımdır. Floyd-Hoare mantığı olarak da bilinen bu yaklaşım, ön koşul (precondition) ve son koşul (postcondition) çiftleriyle deyimlerin anlabilimini tanımlar ve programların matematiksel olarak doğru olduğunu kanıtlamayı mümkün kılar.

Soru 19

EN: What two things must be defined for each language entity in order to construct a denotational description of the language?

TR: Dilin gösterimsel tanımını oluşturmak için her dil varlığı için hangi iki şey tanımlanmalıdır?

● **Cevap / Yönlendirme**

Gösterimsel anlambilimde her dil varlığı için tanımlanması gereken iki şey: (1) Sözdizimsel alanı — dil varlığının sözdizimsel bileşenlerini tanımlayan gramer kuralları; (2) Anlambilim eşleme fonksiyonu (semantic mapping function) — sözdizimsel alandan anlambilimsel alana eşlemeyi gerçekleştiren matematiksel fonksiyon. Bu fonksiyon, her sözdizimsel yapıya karşılık gelen matematiksel anlamı verir.

Soru 20

EN: Which part of an inference rule is the antecedent?

TR: Çıkarım kuralının (inference rule) hangi bölümü öncül (antecedent) olarak adlandırılır?

● **Cevap / Yönlendirme**

Çıkarım kuralında öncül (antecedent / hypothesis), çizginin üzerinde yer alan kısımdır. Sonuç (consequent / conclusion) ise çizginin altındaki kısımdır. Öncüller, sonucun geçerli olabilmesi için sağlanması gereken ön koşulları ifade eder. Öncülü olmayan bir kural aksiyom (axiom) olarak adlandırılır; koşulsuz biçimde doğrudur.

Soru 21

EN: What is a predicate transformer function?

TR: Yüklem dönüştürücü fonksiyon (predicate transformer function) nedir?

● **Cevap / Yönlendirme**

Yüklem dönüştürücü fonksiyon (predicate transformer), aksiyomatik anlambilimde bir deyim ve sonraki durumun özelliğini tanımlayan son koşul (postcondition) verildiğinde, o deyim doğru şekilde sonlanabilmesi için giriş durumunun sağlaması gereken en zayıf ön koşulu (weakest precondition) hesaplayan fonksiyondur. $wp(S, Q)$ gösterimiyle ifade edilir.

Soru 22

EN: What does partial correctness mean for a loop construct?

TR: Döngü yapısı için kısmi doğruluk (partial correctness) ne anlama gelir?

● **Cevap / Yönlendirme**

Kısmi doğruluk (partial correctness), bir döngünün eğer sonlanırsa doğru sonuç vereceği anlamına gelir; ancak döngünün kesinlikle sonlanacağını garanti etmez. Tam doğruluk (total correctness) ise hem kısmi doğruluğu hem de sonlanma (termination) garantisini kapsar. Pratikte döngülerin doğruluğunu kanıtlarken kısmi ve tam doğruluk ayrı ayrı ele alınır.

Soru 23

EN: On what branch of mathematics is axiomatic semantics based?

TR: Aksiyomatik anlambilim (axiomatic semantics) matematiğin hangi dalına dayanır?

● Cevap / Yönlendirme

Aksiyomatik anlambilim, matematiksel mantık (mathematical logic) — özellikle yüklem mantığı (predicate logic) ve formal ispat sistemleri (formal proof systems) — üzerine inşa edilmiştir. Floyd-Hoare mantığı olarak da bilinen bu yaklaşım, Hoare üçlüleri $\{P\} S \{Q\}$ biçiminde ifadeler kullanarak program doğruluğunu matematiksel olarak kanıtlamayı sağlar.

Soru 24

EN: On what branch of mathematics is denotational semantics based?

TR: Gösterimsel anlambilim (denotational semantics) matematiğin hangi dalına dayanır?

● Cevap / Yönlendirme

Gösterimsel anlambilim, lambda hesabı (lambda calculus) ve özellikle Scott-Strachey tarafından geliştirilen örgü teorisi (domain theory / lattice theory) üzerine inşa edilmiştir. Program yapıları sürekli fonksiyonlar olarak modellenir; özyinelemeli tanımlar sabit nokta teorisi (fixed-point theory) kullanılarak çözümlenir.

Soru 25

EN: What is the problem with using a software pure interpreter for operational semantics?

TR: İşlemsel anlambilim için yazılım saf yorumlayıcı kullanmanın sorunu nedir?

● Cevap / Yönlendirme

Yazılım saf yorumlayıcısının kullandığı sanal makine, yeterince resmi (formal) değilse anlambilim tanımı güvenilir hâle gelmez. Yorumlayıcının davranışı dilin anlambilimini tanımlamak yerine yalnızca onu uyguluyor olabilir ve bu da döngüsel (circular) bir tanıma yol açar. Ayrıca yorumlayıcı hataları dilin anlambilimini yanlış tanımlar; formal matematiksel araçların sağladığı kesinliği sunamaz.

Soru 26

EN: Explain what the preconditions and postconditions of a given statement mean in axiomatic semantics.

TR: Aksiyomatik anlambilimde bir deyimın ön koşul (precondition) ve son koşulunun (postcondition) ne anlama geldiğini açıklayın.

● Cevap / Yönlendirme

Son koşul (postcondition), deyim başarıyla yürütüldükten sonra program durumunun sağlaması gereken özellikleri tanımlayan bir mantıksal önermedir. Ön koşul (precondition), son koşulun sağlanabilmesi için deyim yürütülmeden önce program durumunun sağlaması gereken koşulları belirtir. En zayıf ön koşul (weakest precondition), son koşulun sağlanması için yeterli olan en az kısıtlayıcı ön koşuldur.

Soru 27

EN: Describe the approach of using axiomatic semantics to prove the correctness of a given program.

TR: Bir programın doğruluğunu kanıtlamak için aksiyomatik anlambilim kullanma yaklaşımını açıklayın.

● Cevap / Yönlendirme

Aksiyomatik anlambilimle program doğrulama şu adımları izler: (1) Programın son koşulu (postcondition) belirlenir. (2) Programın sondan başa doğru her deyimi için $wp()$ fonksiyonu uygulanarak en zayıf ön koşul hesaplanır. (3) Programın başına gelindiğinde elde edilen ön koşul, programın başlangıç koşulunu (precondition) ifade etmelidir. (4) Döngüler için döngü değişmezi (loop invariant) belirlenerek ek kanıtlama adımları uygulanır.

Soru 28

EN: Describe the basic concept of denotational semantics.

TR: Gösterimsel anlambilimin (denotational semantics) temel kavramını açıklayın.

● Cevap / Yönlendirme

Gösterimsel anlambilim, her sözdizimsel yapıya matematiksel bir nesne (denotation) atayarak dilin anlambilimini tanımlar. Her dil yapısı için bir anlambilim fonksiyonu (semantic function) tanımlanır; bu fonksiyon sözdizimsel yapıyı alır ve anlambilimsel alanındaki (matematiksel) karşılığını döndürür. Bu yaklaşım, dilin anlambilimini programlama dilinden bağımsız, saf matematiksel terimlerle ifade eder.

Soru 29

EN: In what fundamental way do operational semantics and denotational semantics differ?

TR: İşlemsel anlambilim (operational semantics) ile gösterimsel anlambilim (denotational semantics) temelde nasıl farklılaşır?

● Cevap / Yönlendirme

İşlemsel anlambilim, bir programın nasıl çalıştığını adım adım hesaplama işlemleriyle tanımlar — sürecin kendisine odaklanır ve genellikle durum geçişlerini (state transitions) açıklar. Gösterimsel anlambilim ise programın ne anlama geldiğini matematiksel nesnelerle tanımlar — hesaplama sürecini değil, programın matematiksel değerini (denotation) verir. İşlemsel yaklaşım sürecin kendisine, gösterimsel yaklaşım ise sonucun matematiksel anlamına odaklanır.

BÖLÜM 3 — PROBLEM SETİ (Problem Set) • 28 Soru**Problem 1**

EN: The two mathematical models of language description are generation and recognition. Describe how each can define the syntax of a programming language.

TR: Dil tanımlamanın iki matematiksel modeli üretim (generation) ve tanıma (recognition) olarak bilinir. Her birinin programlama dilinin sözdizimini nasıl tanımlayabileceğini açıklayın.

● Cevap / Yönlendirme

Üretim modeli (generation): Gramer kuralları aracılığıyla, başlangıç sembolünden yola çıkılarak dilin tüm geçerli cümleleri üretilir. BNF/EBNF gramerleri bu yaklaşıma örnektir. Tanıma modeli (recognition): Verilen bir dizinin dile ait olup olmadığını belirler. Bir otomata (automaton) ya da ayrıştırıcı (parser), girdi dizisini dil gramerine göre kabul veya reddeder. Derleyicilerin sözdizimi analiz aşaması bu modeli kullanır.

Problem 2

EN: Write EBNF descriptions for the following: (a) A Java class definition header statement (b) A Java method call statement (c) A C switch statement (d) A C union definition (e) C float literals

TR: Aşağıdakiler için EBNF tanımları yazın: (a) Java sınıf tanım başlığı (b) Java metod çağırısı (c) C switch deyimi (d) C union tanımı (e) C float sabitleri

● Cevap / Yönlendirme

Örnek EBNF tanımları:

- (a) `<class_header> → {modifier} class <id> [extends <id>] [implements <id> {, <id>}]`
- (b) `<method_call> → [<obj>.]<id>(<arg_list>) ; <arg_list> → [<expr> {, <expr>}]`
- (c) `<switch> → switch (<expr>) { {case <const>: {<stmt>}} [default: {<stmt>}]}`
- (d) `<union_def> → union [<id>] { <type> <id>; {<type> <id>;} }`
- (e) `<float> → [+|-] <digit> {<digit>} [.<digit> {<digit>}] [e[+|-]<digit> {<digit>}]`

Problem 3

EN: Rewrite the BNF of Example 3.4 to give + precedence over * and force + to be right associative.

TR: Örnek 3.4'ün BNF'ini + operatörünün *'dan önce gelmesi ve +'nın sağ birleşimli (right associative) olacak şekilde yeniden yazın.

● Cevap / Yönlendirme

Standart hiyerarşide + için daha düşük öncelik seviyesi kullanılır. +'yı daha yüksek öncelikli yapmak için seviyeleri tersine çeviriyoruz:

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{expr} \rangle \mid \langle \text{term} \rangle$ [* sol-birleşimli kalır, daha düşük öncelik]

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle + \langle \text{term} \rangle \mid \langle \text{factor} \rangle$ [+ sağ-birleşimli, daha yüksek öncelik]

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

Bu şekilde + operatörü * operatöründen daha sıkı bağlanır.

Problem 4

EN: Rewrite the BNF of Example 3.4 to add the ++ and -- unary operators of Java.

TR: Örnek 3.4'ün BNF'ini Java'nın ++ ve -- tekli (unary) operatörlerini ekleyecek şekilde yeniden yazın.

● Cevap / Yönlendirme

Tekli ++ ve -- operatörleri en yüksek önceliğe sahip olmalı ve $\langle \text{factor} \rangle$ düzeyine eklenmeli:

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle \mid ++\langle \text{id} \rangle \mid --\langle \text{id} \rangle \mid \langle \text{id} \rangle ++ \mid \langle \text{id} \rangle --$

Ön ek (prefix) ve son ek (postfix) biçimleri ayrı ayrı tanımlanır; her iki form da $\langle \text{factor} \rangle$ düzeyinde yer alır.

Problem 5

EN: Write a BNF description of the Boolean expressions of Java, including the three operators &&, ||, and ! and the relational expressions.

TR: Java'nın &&, ||, ! operatörlerini ve ilişkisel ifadeleri içeren Boolean ifadelerinin BNF tanımını yazın.

● Cevap / Yönlendirme

$\langle \text{bool_expr} \rangle \rightarrow \langle \text{bool_expr} \rangle \mid \mid \langle \text{bool_and} \rangle \mid \langle \text{bool_and} \rangle$

$\langle \text{bool_and} \rangle \rightarrow \langle \text{bool_and} \rangle \&\& \langle \text{bool_not} \rangle \mid \langle \text{bool_not} \rangle$

$\langle \text{bool_not} \rangle \rightarrow ! \langle \text{bool_not} \rangle \mid \langle \text{bool_primary} \rangle$

$\langle \text{bool_primary} \rangle \rightarrow \langle \text{rel_expr} \rangle \mid (\langle \text{bool_expr} \rangle) \mid \text{true} \mid \text{false}$

$\langle \text{rel_expr} \rangle \rightarrow \langle \text{arith_expr} \rangle \langle \text{rel_op} \rangle \langle \text{arith_expr} \rangle$

$\langle \text{rel_op} \rangle \rightarrow == \mid != \mid < \mid > \mid <= \mid >=$

Öncelik sırası: ! (en yüksek) > && > || (en düşük)

Problem 6 [Parse Tree]

EN: Using the grammar in Example 3.2, show a parse tree and a leftmost derivation for each of the following statements: (a) $A = A*(B+(C*A))$ (b) $B = C*(A*C+B)$ (c) $A = A*(B+(C))$

TR: Örnek 3.2'deki grameri kullanarak aşağıdaki deyimler için parse tree ve en soldan türetme (leftmost derivation) gösterin: (a) $A=A*(B+(C*A))$ (b) $B=C*(A*C+B)$ (c) $A=A*(B+(C))$

● Cevap / Yönlendirme

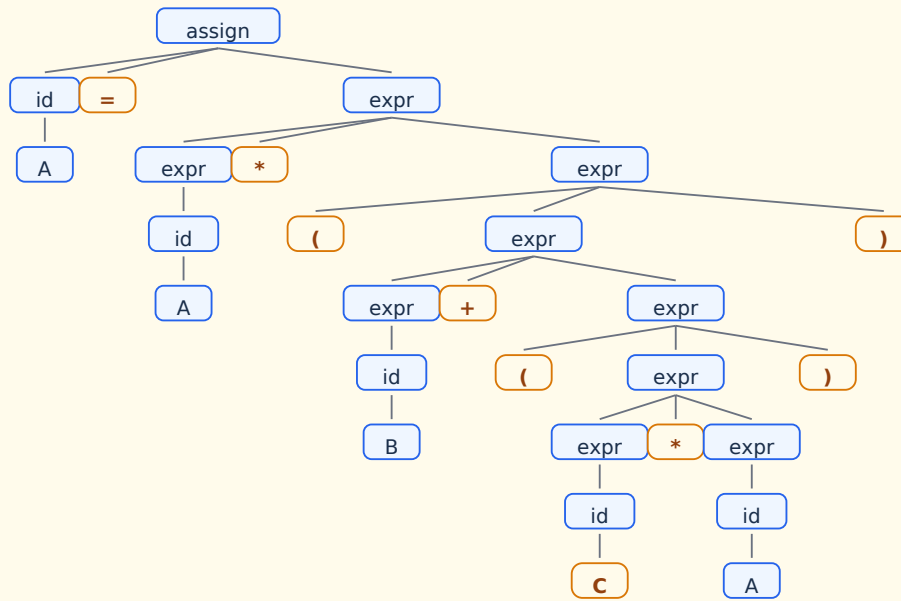
Parse tree diyagramları ve en soldan türetmeler aşağıdaki ağaç görsellerinde ve türetme adımlarında gösterilmektedir. Kullanılan gramer (Örnek 3.2 — belirsiz):

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

□ Parse Tree — 6a: $A = A*(B+(C*A))$

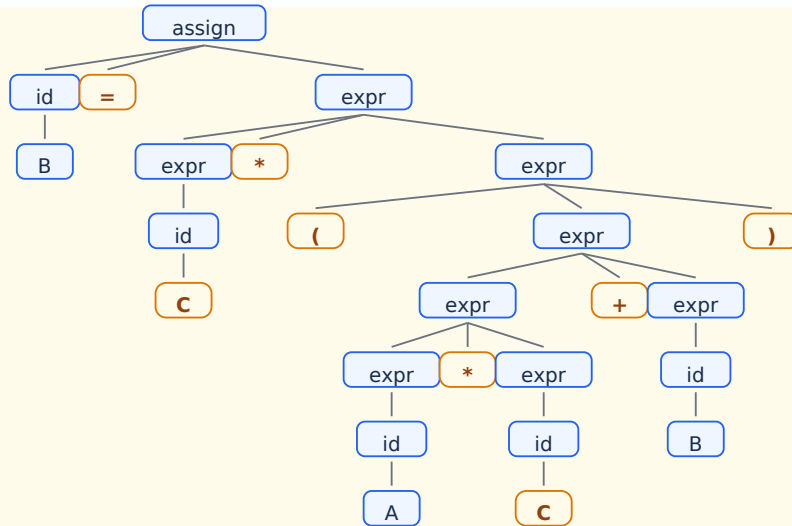


□ En Soldan Türetme — 6a: $A = A*(B+(C*A))$

```

□ =
□ A =
□ A = *
□ A = *
□ A = A *
□ A = A * ( )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( B + )
□ A = A * ( B + ( ) )
□ A = A * ( B + ( * ) )
□ A = A * ( B + ( * ) )
□ A = A * ( B + ( C * ) )
□ A = A * ( B + ( C * ) )
□ A = A * ( B + ( C * A ) )
  
```

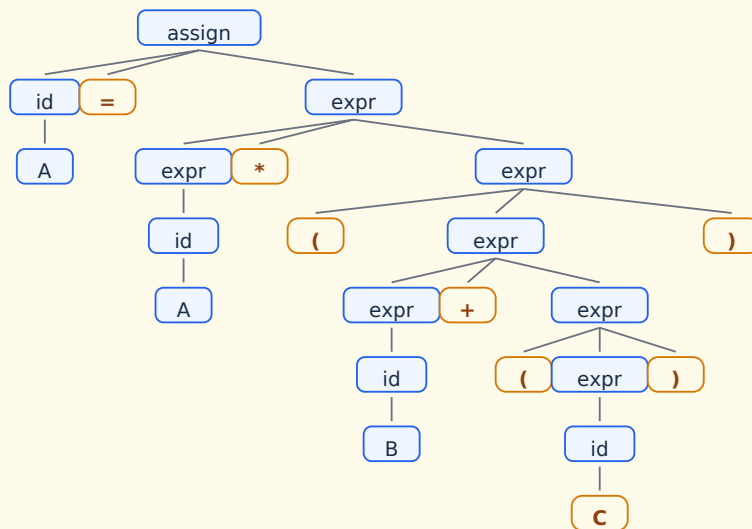
□ Parse Tree — 6b: $B = C*(A*C+B)$



En Soldan Türetme — 6b: $B = C*(A*C+B)$

□ =
 □ B =
 □ B = *
 □ B = *
 □ B = C *
 □ B = C * ()
 □ B = C * (+)
 □ B = C * (* +)
 □ B = C * (* +)
 □ B = C * (A * +)
 □ B = C * (A * +)
 □ B = C * (A * C +)
 □ B = C * (A * C +)
 □ B = C * (A * C + B)

Parse Tree — 6c: $A = A*(B+(C))$



En Soldan Türetme — 6c: $A = A*(B+(C))$

```

□ =
□ A =
□ A = *
□ A = *
□ A = A *
□ A = A * ( )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( B + )
□ A = A * ( B + ( ) )
□ A = A * ( B + ( ) )
□ A = A * ( B + ( C ) )

```

Problem 7 [Parse Tree]

EN: Using the grammar in Example 3.4, show a parse tree and a leftmost derivation for each of the following: (a) $A=(A+B)*C$ (b) $A=B+C+A$ (c) $A=A*(B+C)$ (d) $A=B*(C*(A+B))$

TR: Örnek 3.4'ün gramerini kullanarak aşağıdakiler için parse tree ve en soldan türetme gösterin: (a) $A=(A+B)*C$ (b) $A=B+C+A$ (c) $A=A*(B+C)$ (d) $A=B*(C*(A+B))$

● Cevap / Yönlendirme

Parse tree diyagramları aşağıda gösterilmektedir. Kullanılan gramer (Örnek 3.4 — belirsiz değil):

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

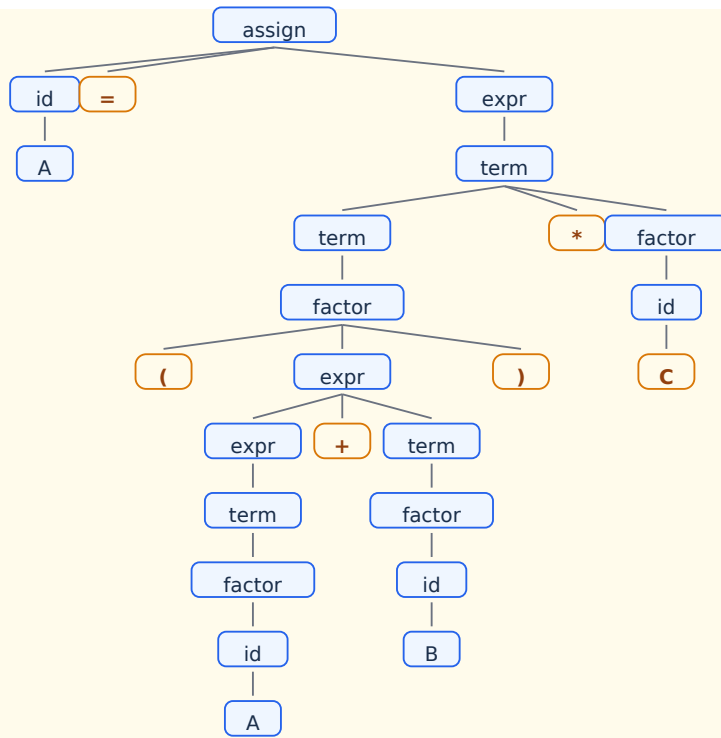
$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

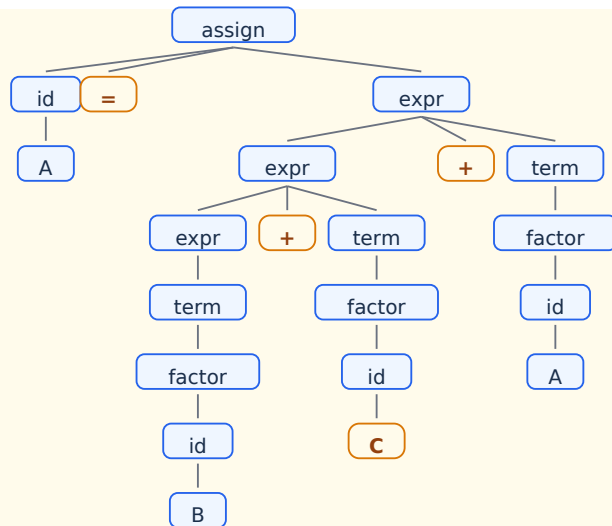
Parse Tree — 7a: $A = (A+B)*C$



□ En Soldan Türetme — 7a: $A = (A+B)*C$

□ =
 □ A =
 □ A =
 □ A = *
 □ A = *
 □ A = () *
 □ A = (+) *
 □ A = (+) *
 □ A = (+) *
 □ A = (+) *
 □ A = (+) *
 □ A = (A +) *
 □ A = (A +) *
 □ A = (A +) *
 □ A = (A + B) *
 □ A = (A + B) *
 □ A = (A + B) * C

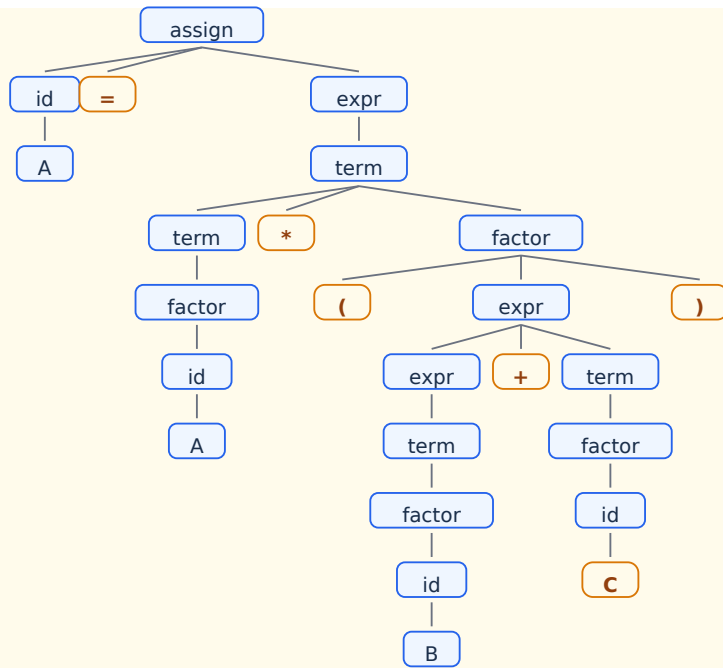
□ Parse Tree — 7b: $A = B+C+A$



□ **En Soldan Türetme** — 7b: $A = B+C+A$

- ☐ =
- ☐ A =
- ☐ A = +
- ☐ A = + +
- ☐ A = + +
- ☐ A = + +
- ☐ A = + +
- ☐ A = B + +
- ☐ A = B + +
- ☐ A = B + +
- ☐ A = B + C +
- ☐ A = B + C +
- ☐ A = B + C +
- ☐ A = B + C + A

Parse Tree — 7c: $A = A*(B+C)$

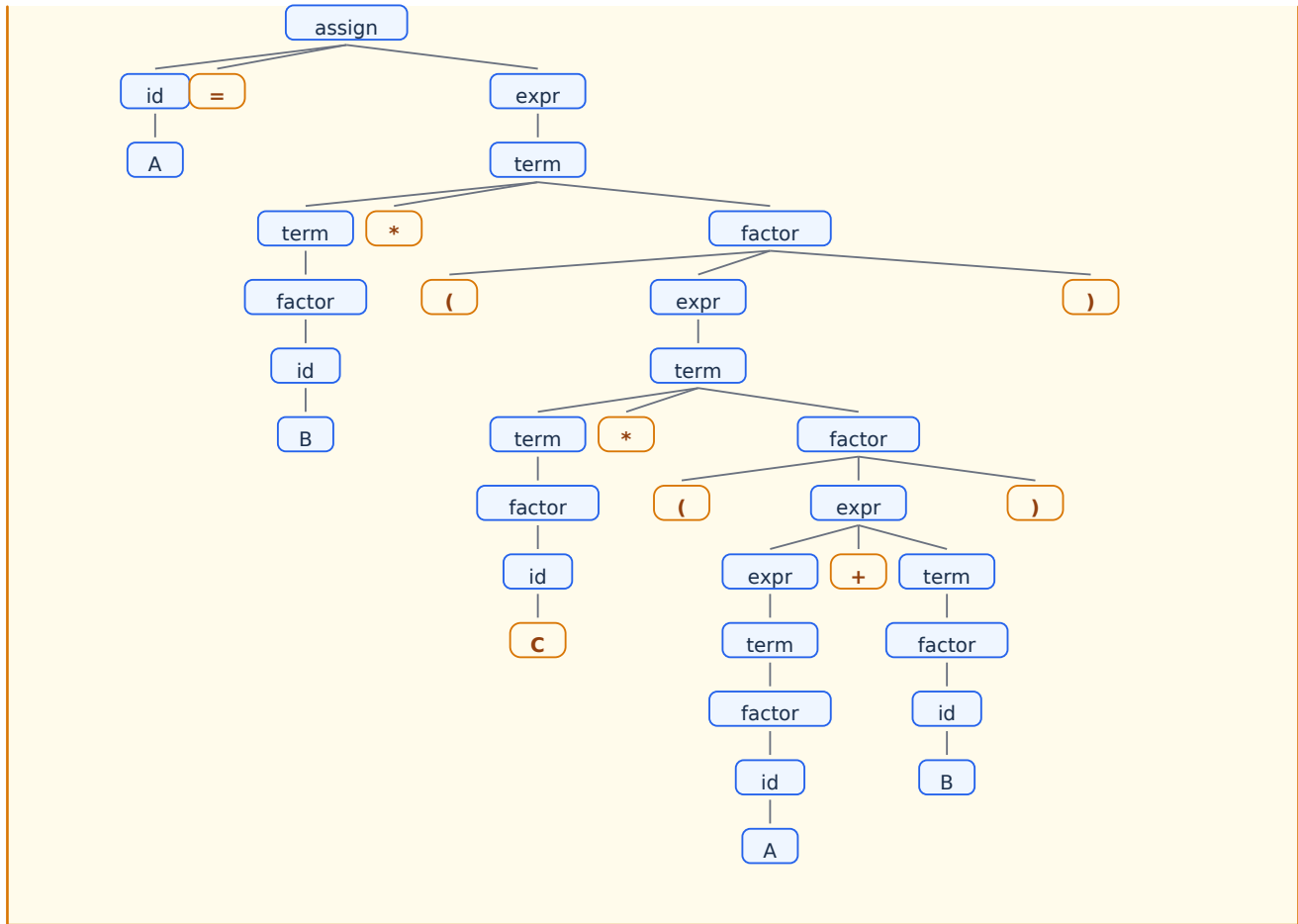


□ En Soldan Türetme — 7c: $A = A*(B+C)$

```

□ =
□ A =
□ A =
□ A = *
□ A = *
□ A = *
□ A = A *
□ A = A * ( )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( + )
□ A = A * ( B + )
□ A = A * ( B + )
□ A = A * ( B + )
□ A = A * ( B + C )
  
```

□ Parse Tree — 7d: $A = B*(C*(A+B))$



En Soldan Türetme — 7d: $A = B*(C*(A+B))$

$A = A =$
 $A = *$
 $A = *$
 $A = *$
 $A = B *$
 $A = B * ()$
 $A = B * ()$
 $A = B * (*)$
 $A = B * (*)$
 $A = B * (*)$
 $A = B * (C *)$
 $A = B * (C * ())$
 $A = B * (C * (+))$
 $A = B * (C * (+))$
 $A = B * (C * (+))$
 $A = B * (C * (+))$
 $A = B * (C * (A +))$
 $A = B * (C * (A +))$
 $A = B * (C * (A +))$
 $A = B * (C * (A + B))$

Problem 8

EN: Prove that the following grammar is ambiguous: $\langle S \rangle \rightarrow \langle A \rangle \langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \mid \langle id \rangle \langle id \rangle \rightarrow a \mid b \mid c$

TR: Aşağıdaki gramerin belirsiz (ambiguous) olduğunu kanıtlayın: $\langle S \rangle \rightarrow \langle A \rangle \langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \mid \langle id \rangle \langle id \rangle \rightarrow a \mid b \mid c$

● Cevap / Yönlendirme

Bir gramerin belirsiz olduğunu kanıtlamak için aynı cümle için iki farklı parse tree'ye (ya da iki farklı leftmost derivation'a) sahip olduğunu göstermek yeterlidir.

$a + b + c$ cümlesi için iki farklı türetme:

Türetme 1: $\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle + \langle A \rangle \rightarrow a + \langle A \rangle + \langle A \rangle \rightarrow a + b + \langle A \rangle \rightarrow a + b + c$ [sol gruplandırma]

Türetme 2: $\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \rightarrow a + \langle A \rangle \rightarrow a + \langle A \rangle + \langle A \rangle \rightarrow a + b + \langle A \rangle \rightarrow a + b + c$ [sağ gruplandırma]
İki farklı ayrıştırma ağacı olduğundan gramer belirsizdir.

Problem 9

EN: Modify the grammar of Example 3.4 to add a unary minus operator that has higher precedence than either $+$ or $*$.

TR: Örnek 3.4'ün gramerini, hem $+$ hem de $*$ 'dan daha yüksek önceliğe sahip tekli eksi (unary minus) operatörü ekleyecek şekilde değiştirin.

● Cevap / Yönlendirme

Tekli eksi operatörü en yüksek önceliğe sahip olacağından $\langle \text{factor} \rangle$ düzeyinin altına, yeni bir $\langle \text{primary} \rangle$ seviyesi ekleyerek:

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow - \langle \text{factor} \rangle \mid \langle \text{primary} \rangle$ [özyinelemeli: $--a$ gibi çiftte tekli eksiye de destekler]

$\langle \text{primary} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

Problem 10

EN: Describe, in English, the language defined by the following grammar: $\langle S \rangle \rightarrow \langle A \rangle \langle B \rangle \langle C \rangle \mid \langle A \rangle \rightarrow a \langle A \rangle \mid a \mid \langle B \rangle \rightarrow b \langle B \rangle \mid b \mid \langle C \rangle \rightarrow c \langle C \rangle \mid c$

TR: Aşağıdaki gramer tarafından tanımlanan dili Türkçe olarak açıklayın: $\langle S \rangle \rightarrow \langle A \rangle \langle B \rangle \langle C \rangle \mid \langle A \rangle \rightarrow a \langle A \rangle \mid a \mid \langle B \rangle \rightarrow b \langle B \rangle \mid b \mid \langle C \rangle \rightarrow c \langle C \rangle \mid c$

● Cevap / Yönlendirme

Bu gramer, a harflerinin bir veya daha fazla kopyasının ardından b harflerinin bir veya daha fazla kopyasının, ardından da c harflerinin bir veya daha fazla kopyasının geldiği tüm dizelerden oluşan dili tanımlar. Daha formal ifadeyle: $L = \{ a^i b^j c^k \mid i \geq 1, j \geq 1, k \geq 1 \}$. Örnek geçerli dizeler: abc , $aabbc$, $aaabbbccc$. Dikkat: i , j ve k birbirinden bağımsızdır; eşit olmak zorunda değildir.

Problem 11

EN: Consider the grammar: $\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$ $\langle A \rangle \rightarrow \langle A \rangle b \mid b$ $\langle B \rangle \rightarrow a \langle B \rangle \mid a$. Which sentences are in the language? (a) baab (b) bbbab (c) bbaaaaaS (d) bbaab

TR: Verilen gramer için hangi dizeler dile aittir? (a) baab (b) bbbab (c) bbaaaaaS (d) bbaab

● Cevap / Yönlendirme

- (a) baab: $\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b \rightarrow ba a b \rightarrow baab \checkmark$ GEÇERLİ ($\langle A \rangle \rightarrow b$, $\langle B \rangle \rightarrow a$)
 (b) bbbab: $\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b \rightarrow \langle A \rangle b a b \rightarrow bbb a b$ — $\langle B \rangle$ yalnızca a^+ üretir; 'b' üretemez $\times$ GEÇERSİZ
 (c) bbaaaaaS: Non-terminal S içerdiğinden cümle değildir $\times$ GEÇERSİZ
 (d) bbaab: $\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b \rightarrow \langle A \rangle b \cdot a \cdot \langle B \rangle \cdot b \rightarrow bb a a b = bbaab \checkmark$ GEÇERLİ ($\langle A \rangle \rightarrow \langle A \rangle b \rightarrow bb$, $\langle B \rangle \rightarrow a$)

Problem 12

EN: Consider the grammar: $\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle \mid \langle A \rangle \mid b$ $\langle A \rangle \rightarrow c \langle A \rangle \mid c$ $\langle B \rangle \rightarrow d \mid \langle A \rangle$. Which sentences? (a) abcd (b) acccbd (c) acccbcc (d) acd (e) accc

TR: Verilen gramer için hangi dizeler dile aittir? (a) abcd (b) acccbd (c) acccbcc (d) acd (e) accc

● Cevap / Yönlendirme

- (a) abcd: $\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle \rightarrow a \cdot b \cdot c \cdot d = abcd \checkmark$ GEÇERLİ
 (b) acccbd: $\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle \rightarrow a \cdot \langle A \rangle \cdot c \cdot \langle B \rangle \rightarrow a \cdot ccc \cdot c \cdot d$ — b harfi çıkmaz $\times$ GEÇERSİZ
 (c) acccbcc: $\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle \rightarrow a \langle S \rangle c \langle A \rangle \rightarrow a \cdot b \cdot c \cdot cc$ — 'b' $\langle S \rangle \rightarrow b$: acccbcc oluşturulamaz $\times$ GEÇERSİZ
 (d) acd: $\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle \rightarrow a \cdot \langle A \rangle \cdot c \cdot d \rightarrow a \cdot c \cdot c \cdot d = accd \neq acd \times$ GEÇERSİZ
 (e) accc: $\langle S \rangle \rightarrow \langle A \rangle \rightarrow c \langle A \rangle \rightarrow cc \langle A \rangle \rightarrow ccc$ — 'a' harfi yok $\times$ GEÇERSİZ; $\langle S \rangle \rightarrow \langle A \rangle$ ile sadece c^+ üretilir

Problem 13

EN: Write a grammar for the language of strings with n copies of 'a' followed by n copies of 'b' ($n > 0$). Example: ab, aaaabbbb are in; a, abb, ba are not.

TR: n adet 'a' harfinin ardından n adet 'b' harfinin geldiği ($n > 0$) dizelerden oluşan dil için bir gramer yazın.

● Cevap / Yönlendirme

Bu klasik $a^n b^n$ dili, bağlamdan bağımsız (context-free) ancak düzenli (regular) olmayan bir dildir:

$\langle S \rangle \rightarrow a \langle S \rangle b \mid ab$

Alternatif olarak:

$\langle S \rangle \rightarrow a \langle S \rangle b \mid a b$

Doğrulama:

- ab : $\langle S \rangle \rightarrow ab$ ✓
- $aabb$: $\langle S \rangle \rightarrow a \langle S \rangle b \rightarrow a \cdot ab \cdot b = aabb$ ✓
- $aaaabbbb$: $\langle S \rangle \rightarrow a \langle S \rangle b \rightarrow a \cdot a \langle S \rangle b \cdot b \rightarrow a \cdot a \cdot ab \cdot b \cdot b = aaaabbbb$ ✓
- Herhangi bir $i \neq j$ için $a^i b^j$: gramer bu dizeyi üretemez ✗

Problem 14 [Parse Tree]

EN: Draw parse trees for the sentences $aabb$ and $aaaabbbb$, as derived from the grammar of Problem 13.

TR: Problem 13'teki gramerden türetilen $aabb$ ve $aaaabbbb$ cümleleri için parse tree çizin.

● Cevap / Yönlendirme

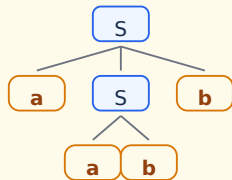
Gramer: $\langle S \rangle \rightarrow a \langle S \rangle b \mid ab$

$aabb$ için: $\langle S \rangle \rightarrow a \langle S \rangle b \rightarrow a \cdot ab \cdot b = aabb$

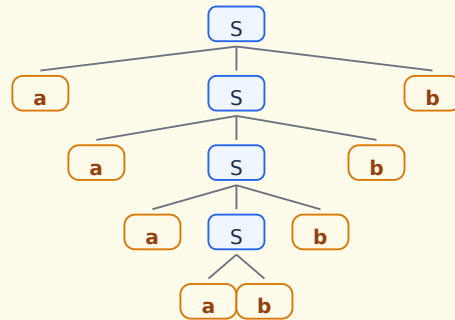
$aaaabbbb$ için: $\langle S \rangle \rightarrow a \langle S \rangle b \rightarrow a \cdot a \langle S \rangle b \cdot b \rightarrow a \cdot a \cdot a \langle S \rangle b \cdot b \cdot b \rightarrow a \cdot a \cdot a \cdot ab \cdot b \cdot b \cdot b = aaaabbbb$

Ağaç diyagramları aşağıda verilmiştir.

□ Parse Tree — $aabb$



□ Parse Tree — $aaaabbbb$



Problem 15

EN: Convert the BNF of Example 3.1 to EBNF.

TR: Örnek 3.1'in BNF'ini EBNF'e dönüştürün.

● Cevap / Yönlendirme

Örnek 3.1 (tipik bir if-then-else BNF tanımı) için EBNF dönüşümü:

BNF: $\langle \text{if_stmt} \rangle \rightarrow \text{if } (\langle \text{expr} \rangle) \langle \text{stmt} \rangle \mid \text{if } (\langle \text{expr} \rangle) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

EBNF: $\langle \text{if_stmt} \rangle \rightarrow \text{if } (\langle \text{expr} \rangle) \langle \text{stmt} \rangle [\text{else } \langle \text{stmt} \rangle]$

BNF: $\langle \text{stmt_list} \rangle \rightarrow \langle \text{stmt} \rangle \mid \langle \text{stmt_list} \rangle ; \langle \text{stmt} \rangle$

EBNF: $\langle \text{stmt_list} \rangle \rightarrow \langle \text{stmt} \rangle \{ ; \langle \text{stmt} \rangle \}$

EBNF, seçenekleri [] ve {} ile daha kısa ifade eder; tekrar eden kuralları ortadan kaldırır.

Problem 16

EN: Convert the BNF of Example 3.3 to EBNF.

TR: Örnek 3.3'ün BNF'ini EBNF'e dönüştürün.

● Cevap / Yönlendirme

Örnek 3.3 (tipik bir ifade grameri) için EBNF dönüşümü:

BNF:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

EBNF:

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ + \langle \text{term} \rangle \}$

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \{ * \langle \text{factor} \rangle \}$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

EBNF'de $\{ + \langle \text{term} \rangle \}$ ifadesi sol özyinelemeyi ortadan kaldırarak sıfır veya daha fazla toplama işlemini kompakt biçimde tanımlar.

Problem 17

EN: Convert the following EBNF to BNF: $S \rightarrow A\{bA\}$ $A \rightarrow a[b]A$

TR: Aşağıdaki EBNF'i BNF'e dönüştürün: $S \rightarrow A\{bA\}$ $A \rightarrow a[b]A$

● Cevap / Yönlendirme

EBNF $\rightarrow$ BNF dönüşümü (yeni yardımcı non-terminal'ler eklenerek):

$S \rightarrow A\{bA\}$ için:

$S \rightarrow A S'$

$S' \rightarrow b A S' \mid \epsilon$ [ϵ = boş dize]

$A \rightarrow a[b]A$ için:

$A \rightarrow a A' A$

$A' \rightarrow b \mid \epsilon$

Toplu BNF:

$S \rightarrow A S'$

$S' \rightarrow b A S' \mid \epsilon$

$A \rightarrow a A' A$

$A' \rightarrow b \mid \epsilon$

Problem 18

EN: What is the difference between an intrinsic attribute and a nonintrinsic synthesized attribute?

TR: İçsel (intrinsic) öznitelik ile içsel olmayan sentezlenmiş (nonintrinsic synthesized) öznitelik arasındaki fark nedir?

● Cevap / Yönlendirme

İçsel öznitelik (intrinsic attribute), başlangıçta ayrıştırma ağacının dışından elde edilen değeri önceden tanımlanmış olan özniteliktir — örneğin sayı sabitleri için tür özniteliği doğrudan sözdizimsel formdan bilinir ($5 \rightarrow \text{integer}$). İçsel olmayan sentezlenmiş öznitelik ise değeri alt düğümlerin özniteliklerinden hesaplama yoluyla elde edilen özniteliktir. İçsel öznitelikler dışarıdan gelirken, içsel olmayan öznitelikler ağaç içinde hesaplanır.

Problem 19

EN: Write an attribute grammar for data type rules: types cannot be mixed in expressions, but assignment need not have same types on both sides.

TR: Veri tipi kuralları için öznitelik grameri yazın: İfadelerde tipler karıştırılamaz, ancak atama işleminde her iki tarafın aynı tipte olması gerekmez.

● Cevap / Yönlendirme

Öznitelik grameri (Örnek 3.6 tabanlı):

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

Kural: (tür kontrolü yok — sadece iç tutarlılık)

$\langle \text{expr} \rangle \rightarrow \langle \text{expr2} \rangle + \langle \text{expr3} \rangle$

$\text{actual_type}(\langle \text{expr} \rangle) = \text{if } \text{type}(\langle \text{expr2} \rangle) == \text{type}(\langle \text{expr3} \rangle)$

then $\text{type}(\langle \text{expr2} \rangle)$

else type_error

Yüklem: $\text{actual_type}(\langle \text{expr} \rangle) \neq \text{type_error}$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle$

$\text{actual_type}(\langle \text{expr} \rangle) = \text{lookup}(\text{id.name})$

Atamada iki taraf farklı tiplere sahip olabilir; yalnızca ifade içinde tip karışımı yasaktır.

Problem 20

EN: Write an attribute grammar whose base BNF is that of Example 3.2 and whose type rules are the same as for the assignment statement example of Section 3.4.5.

TR: Taban BNF'i Örnek 3.2'den alınan ve tür kuralları Bölüm 3.4.5'teki atama deyimi örneğiyle aynı olan bir öznitelik grameri yazın.

● Cevap / Yönlendirme

Örnek 3.2 BNF'i ile öznitelik grameri:

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

Yüklem: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{id} \rangle.\text{actual_type}$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr2} \rangle + \langle \text{expr3} \rangle$

$\langle \text{expr} \rangle.\text{actual_type} =$

if $\langle \text{expr2} \rangle.\text{actual_type} == \text{int}$ AND $\langle \text{expr3} \rangle.\text{actual_type} == \text{int}$

then int

else if $\langle \text{expr2} \rangle.\text{actual_type} == \text{real}$ OR $\langle \text{expr3} \rangle.\text{actual_type} == \text{real}$

then real else type_error

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle$

$\langle \text{expr} \rangle.\text{actual_type} = \text{lookup}(\langle \text{id} \rangle.\text{name})$

$\langle \text{id} \rangle \rightarrow A|B|C$ $\text{actual_type} =$ sembol tablosundan alınır

Problem 21

EN: Using the virtual machine instructions, give an operational semantic definition of: (a) Java do-while (b) Ada for (c) C++ if-then-else (d) C for (e) C switch

TR: Sanal makine komutlarını kullanarak aşağıdakilerin işlemsel anlambilim tanımını verin: (a) Java do-while (b) Ada for (c) C++ if-then-else (d) C for (e) C switch

● Cevap / Yönlendirme

- (a) do-while: loop: <body_code>; eval <expr>; if true goto loop
 (b) Ada for: i=low; loop: if i>high goto exit; <body>; i=i+1; goto loop; exit:
 (c) if-then-else: eval <cond>; if false goto else_part; <then_body>; goto end; else_part: <else_body>; end:
 (d) C for: <init>; loop: eval <cond>; if false goto exit; <body>; <incr>; goto loop; exit:
 (e) switch: eval <expr>; compare case values; jump to matching case; default: if no match

Problem 22

EN: Write a denotational semantics mapping function for: (a) Ada for (b) Java do-while (c) Java Boolean (d) Java for (e) C switch

TR: Aşağıdakiler için gösterimsel anlambilim eşleme fonksiyonu yazın: (a) Ada for (b) Java do-while (c) Java Boolean (d) Java for (e) C switch

● Cevap / Yönlendirme

- (a) Ada for: $M_for(\text{for } i \text{ in } low..high \text{ loop } S \text{ end, } s) = \text{if } s(low) > s(high) \text{ then } s \text{ else } M_for(\text{for } \dots \text{ loop } S, M_s(S, s[i \rightarrow s(low)+1]))$
 (b) Java do-while: $M_dw(\text{do } S \text{ while}(B), s) = M_dw_h(M_s(S, s), B, S)$
 (c) Boolean: $M_bool(E1 \ \&\& \ E2, s) = M_bool(E1, s) \text{ AND } M_bool(E2, s)$
 (d) Java for: $M_for(\text{for}(l; C; U) S, s) = M_s(l; \text{while}(C)\{S; U\}, s)$
 (e) C switch: $M_sw(\text{switch}(E)\{\text{cases}\}, s) = \text{apply matching case body with } M_s$

Problem 23

EN: Compute the weakest precondition for each assignment statement and postcondition: (a) $a=2*(b-1)-1 \ \{a>0\}$ (b) $b=(c+10)/3 \ \{b>6\}$ (c) $a=a+2*b-1 \ \{a>1\}$ (d) $x=2*y+x-1 \ \{x>11\}$

TR: Aşağıdaki atama deyimleri ve son koşulları için en zayıf ön koşulu hesaplayın.

● Cevap / Yönlendirme

wp hesaplama: son koşuldaki değişkeni deyimın sağ tarafıyla değiştirin.

- (a) $a=2*(b-1)-1, \{a>0\}$:
 $wp: 2*(b-1)-1 > 0 \rightarrow 2b-3 > 0 \rightarrow b > 3/2 \rightarrow \{b > 1\}$
 (b) $b=(c+10)/3, \{b>6\}$:
 $wp: (c+10)/3 > 6 \rightarrow c+10 > 18 \rightarrow \{c > 8\}$
 (c) $a=a+2*b-1, \{a>1\}$:
 $wp: a+2*b-1 > 1 \rightarrow \{a+2*b > 2\}$
 (d) $x=2*y+x-1, \{x>11\}$:
 $wp: 2*y+x-1 > 11 \rightarrow \{2*y+x > 12\}$

Problem 24

EN: Compute the weakest precondition for sequences: (a) $a=2*b+1; b=a-3 \ \{b<0\}$ (b) $a=3*(2*b+a); b=2*a-1 \ \{b>5\}$

TR: Aşağıdaki deyim dizileri için en zayıf ön koşulu hesaplayın.

● Cevap / Yönlendirme

Diziler için arka plandan öne doğru wp hesaplanır:

(a) $a=2*b+1; b=a-3, \{b<0\}$:

Adım 1: $wp(b=a-3, \{b<0\}) \rightarrow a-3<0 \rightarrow \{a<3\}$

Adım 2: $wp(a=2*b+1, \{a<3\}) \rightarrow 2*b+1<3 \rightarrow 2*b<2 \rightarrow \{b<1\}$

En zayıf ön koşul: $\{b < 1\}$

(b) $a=3*(2*b+a); b=2*a-1, \{b>5\}$:

Adım 1: $wp(b=2*a-1, \{b>5\}) \rightarrow 2*a-1>5 \rightarrow \{a>3\}$

Adım 2: $wp(a=3*(2*b+a), \{a>3\}) \rightarrow 3*(2*b+a)>3 \rightarrow 6b+3a>3 \rightarrow \{6b+3a>3\}$

Problem 25

EN: Compute the weakest precondition for selection constructs: (a) *if*($a==b$) $b=2*a+1$ *else* $b=2*a$ $\{b>1\}$
 (b) *if*($x<y$) $x=x+1$ *else* $x=3*x$ $\{x<0\}$ (c) *if*($x>y$) $y=2*x+1$ *else* $y=3*x-1$ $\{y>3\}$

TR: Aşağıdaki seçim yapıları için en zayıf ön koşulu hesaplayın.

● Cevap / Yönlendirme

Koşul deyimi için: $wp(\text{if } B \text{ then } S1 \text{ else } S2, Q) = (B \rightarrow wp(S1, Q)) \wedge (\neg B \rightarrow wp(S2, Q))$

(a) $\{b>1\}$:

$wp(b=2*a+1, \{b>1\}) = \{2*a+1>1\} = \{a>0\}$

$wp(b=2*a, \{b>1\}) = \{2*a>1\} = \{a>1/2\}$ yani $\{a\geq 1\}$ (tam sayı)

Ön koşul: $(a==b \wedge a>0) \vee (a\neq b \wedge a\geq 1) \rightarrow \{a>0\}$

(b) $\{x<0\}$:

$wp(x=x+1, \{x<0\}) = \{x<-1\}; wp(x=3*x, \{x<0\}) = \{x<0\}$

Ön koşul: $(x<y \wedge x<-1) \vee (x\geq y \wedge x<0)$

(c) $\{y>3\}$:

$wp(y=2*x+1, \{y>3\}) = \{x>1\}; wp(y=3*x-1, \{y>3\}) = \{x>4/3\}$

Ön koşul: $(x>y \wedge x>1) \vee (x\leq y \wedge x>4/3)$

Problem 26

EN: Explain the four criteria for proving the correctness of a logical pretest loop construct: *while* B *do* S *end*.

TR: *while* B *do* S *end* biçimindeki mantıksal ön-test döngü yapısının doğruluğunu kanıtlamanın dört kriterini açıklayın.

● Cevap / Yönlendirme

while döngüsünün doğruluğunu kanıtlamak için dört kriter:

- (1) Döngü değişmezi başlangıçta doğrudur: Döngüye girilmeden önce döngü değişmezinin (loop invariant — I) geçerli olduğu gösterilir.
- (2) Gövde değişmezi korur: $\{I \wedge B\} S \{I\}$ — döngü gövdesi yürütüldüğünde koşul B doğru ve değişmez I geçerliyse, gövde yürütüldükten sonra I hâlâ geçerlidir.
- (3) Sonuç doğrudur: Döngü bittiğinde (B yanlış) $I \wedge \neg B$ son koşulu (postcondition) sağlamalıdır.
- (4) Döngü sonlanır: Döngünün sonlu adımda durduğu kanıtlanır (genellikle azalan bir tamsayı fonksiyonu — variant — kullanılır).

Problem 27

EN: Prove that $(n+1) \cdot n = 1$.

TR: $(n+1) \cdot n = 1$ ifadesini kanıtlayın.

● Cevap / Yönlendirme

Bu ifade Gauss toplam formülüyle ilişkilidir:

İddia: $\sum_{i=1}^n i = n(n+1)/2$

Alternatif gösterim: $1+2+\dots+n = n(n+1)/2$

Mathematiksel tümevarım (induction) ile kanıt:

Temel durum: $n=1 \rightarrow 1 = 1 \cdot 2/2 = 1 \checkmark$

Tümevarım adımı: $n=k$ için doğru kabul edelim: $1+\dots+k = k(k+1)/2$

$1+\dots+k+(k+1) = k(k+1)/2 + (k+1) = (k+1)(k+2)/2 = (k+1)((k+1)+1)/2 \checkmark$

Dolayısıyla $n(n+1)/2$ ifadesi n'den 1'e kadar olan tam sayıların toplamını verir.

Problem 28

EN: Prove the following program is correct: $\{n>0\}$ count=n; sum=0; while count<>0 do sum=sum+count; count=count-1 end $\{sum = 1+2+\dots+n\}$

TR: Aşağıdaki programın doğruluğunu kanıtlayın: $\{n>0\}$ count=n; sum=0; while count<>0 do sum=sum+count; count=count-1 end $\{sum=1+2+\dots+n\}$

● Cevap / Yönlendirme

Döngü değişmezi (Loop Invariant — I): $sum + (1+2+\dots+count) = 1+2+\dots+n$ yani $sum = n(n+1)/2 - count(count+1)/2$

(1) Başlangıç: count=n, sum=0 $\rightarrow 0 + (1+\dots+n) = 1+\dots+n \checkmark$

(2) Korunum: $\{I \wedge count \neq 0\}$ yürütme: $sum \leftarrow sum+count, count \leftarrow count-1$

Yeni sum + $(1+\dots+yeni_count) = (sum+count) + (1+\dots+count-1) = sum+(1+\dots+count) = 1+\dots+n \checkmark$

(3) Sonuç: count=0 olduğunda $I \wedge count=0 \rightarrow sum+(1+\dots+0)=sum=1+\dots+n \checkmark$

(4) Sonlanma: count her adımda 1 azalır; $n>0$ ve count başlangıçta n iken n adımda 0'a ulaşır $\checkmark$